

calc3 — 项目架构

用 C3 编写的学习用计算器：词法分析器、递归下降解析、AST 以及通过访问者遍历树。

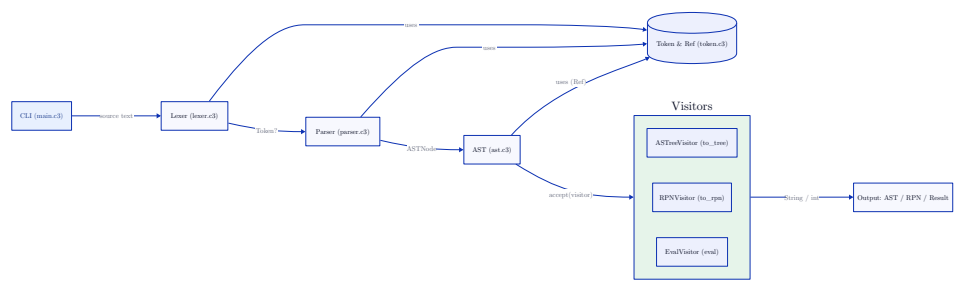
概述

程序接收算术表达式文本，并生成三种表示：AST（树）、RPN（逆波兰表示法）和计算所得的值。数据流是经典的编译器流水线：

源码 -> 词法分析器 -> 语法分析器 -> AST --(accept)--> 访问者 -> 输出

所有阶段共享位置类型 token::Ref { row, col }，因此错误消息会携带源码坐标。

数据流



模块

模块	职责
token.c3	词法单元类型 (TokenKind)、词法单元值 (Value)、位置 (Ref)，以及共享的错误格式化辅助函数 format_ref_error。
lexer.c3	将源码文本转换为词法单元流 (Lexer.next)，跟踪 Ref，并捕获未知字符与整数溢出。
ast.c3	树节点 (ASTNumber、ASTUnary、ASTBinary)、ASTNode/ASTVisitor 接口、工厂函数以及演示用的 ASTestVisitor。
ast_tree.c3	访问者 ASTreeVisitor + to_tree —— 将 AST 渲染为树状结构（风格类似 eza -T）。
rpn_visitor.c3	访问者 RPNVisitor + to_rpn —— 生成逆波兰表示法（一元运算符对应 NEG/POS）。
eval_visitor.c3	访问者 EvalVisitor + eval —— 计算值，返回 Result{int, EvalError}。
parser.c3	基于语法的递归下降 (parse)，构建 AST，并报告带位置的 ParseError。
main.c3	入口点：命令行参数解析、REPL、文件读取以及组装最终输出。

词法单元表示 (token.c3)

- TokenKind 是一个枚举：NUMBER、PLUS、MINUS、MUL、DIV、MOD、LPAREN、RPAREN、EOF，以及错误值 UNKNOWN_CHAR、OVERFLOW。
- Token 为 { kind, val: Value, ref: Ref }，其中 Value 是联合体（数字或字符）。
- Ref 为 { int row, int col }，是错误坐标的唯一来源。
- format_ref_error(String kind, Ref ref, String msg) 是共享模板 "<kind>(<msg> at <row>:<col>)"，被 ParseError 与 EvalError 共同使用。

词法分析器 (lexer.c3)

- Lexer 持有 input、ref 与 pos；eat 更新 ref（包括换行与制表符的处理）。
- next() -> Token?: 跳过空白，根据首字符通过 switch 派发并生成词法单元；对数字调用 eat_int（捕获溢出）。出错时返回错误值（UNKNOWN_CHAR~ / OVERFLOW~），而非抛出异常。

AST 与访问者模式 (ast.c3)

- ASTNode 接口要求实现 accept(ASTVisitor) 与 to_format（用于 Printable）。
- ASTVisitor 接口包含 visit_number / visit_unary / visit_binary。
- 节点：ASTNumber { ref, value }、ASTUnary { ref, op, operand }、ASTBinary { ref, op, left, right }。每个 accept 会调用对应的 visit_*。
- 工厂函数 create_number / create_unary / create_binary 分配节点并以 ASTNode 返回。op_symbol 将运算符的 TokenKind 映射为符号。
- ASTestVisitor 是一个演示用访问者，打印节点结构（用于测试）。

访问者

每个访问者都实现 ASTVisitor，并将结果累积在 DString out 中（对 EvalVisitor 而言则是 int? value / EvalError? error）：

- ASTreeVisitor → to_tree(ASTNode, Allocator) -> String —— 带缩进与方框绘制的树。
- RPNVisitor → to_rpn(ASTNode, Allocator) -> String —— 后缀表示法。
- EvalVisitor → eval(ASTNode, Allocator) -> Result{int, EvalError} —— 表达式的值。

三者都返回字符串的**所有权副本**（str_view().copy(allocator)），这样调用方就不会持有指向访问者内部缓冲区的悬空引用。

语法分析器 (parser.c3)

- parse(String src, Allocator allocator) -> Result{ASTNode, ParseError} —— 递归下降：expression → term → factor → primary，遵循优先级与结合性，并支持一元 +/- 与括号。
- ParseError 持有 Ref 与消息；其 to_format 委托给 token::format_ref_error("ParseError", ...)。

错误处理

两种相互独立的错误类型，各自带有 to_format：

- ParseError —— 解析阶段的错误位置与原因（例如 ParseError(Unexpected character at 1:4)）。
- EvalError —— 计算阶段的错误位置与原因（Division by zero，以及溢出，包括 INT_MIN / -1）。

共享的 token::format_ref_error 保证格式一致且带有坐标。

入口点与命令行 (main.c3)

pipeline(String src, Allocator) -> Result{PipelineOk, ParseError} 将 parse + to_rpn + eval 组合在一起，在一次遍历中返回节点、RPN 与结果。在其之上：

- process_line(src, allocator) —— 简短的 RPN\n 结果 输出（用于测试）；输入 ? 时返回 42。
- process_to_string(src, allocator) —— 完整的 AST / RPN / Result 块。
- process(src) —— 在内存池 (@pool + tmem) 中打印 process_to_string 的结果。

命令行参数解析被抽取到 `parse_args` 中（通过指针形式的出参填充状态，并报告是否继续），而各运行模式则位于 `process_file`（读取文件、按行拆分、逐行处理）与 `run_repl`（基于 `io::treadline` 的 `stdin` 循环）中。`main` 仍是一个轻量调度器：

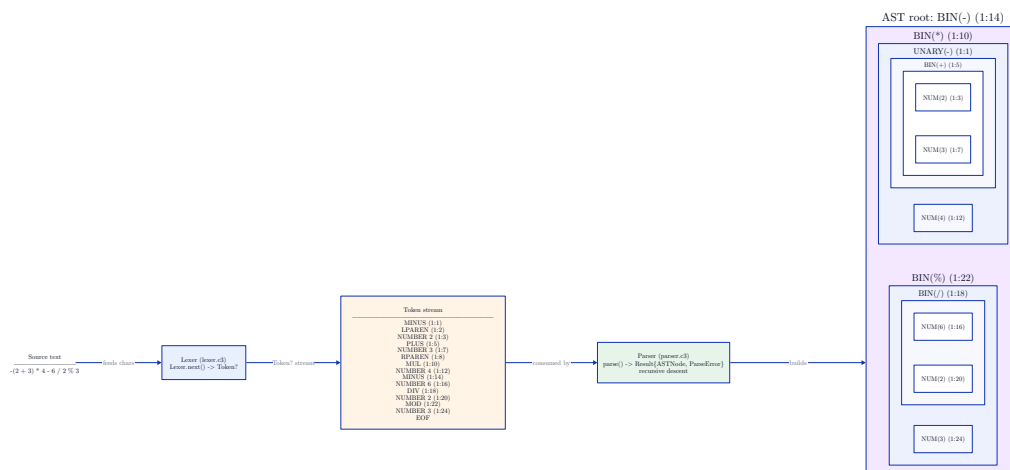
- `calc3` —— 交互式 REPL；
- `calc3 "<表达式>"` —— 单个表达式；
- `calc3 -f <文件> / --file <文件>` —— 从文件读取表达式（每行一个）；
- `calc3 -h / --help` —— 显示帮助；
- `--` —— 分隔符，用于传入以 `-` 开头的表达式（例如 `-5`）。

分配器与所有权

中层函数（`parse`、`to_tree`、`to_rpn`、`eval`、`pipeline`）接受 `Allocator` 并返回所有权副本。顶层（`process`、`process_line`、`process_to_string`）运行在 `@pool` 与临时分配器 `tmem` 中，从而在构建输出后简化内存释放。

示例：从文本到 AST

以测试中的表达式为例：`-(2 + 3) * 4 - 6 / 2 % 3`。



词法分析器将源码转换为词法单元流（每个都带有自己的 `Ref`）：`MINUS (1:1)`、`LPAREN (1:2)`、`NUMBER 2 (1:3)`、`PLUS (1:5)`、`NUMBER 3 (1:7)`、`RPAREN (1:8)`、`MUL (1:10)`、`NUMBER 4 (1:12)`、`MINUS (1:14)`、`NUMBER 6 (1:16)`、`DIV (1:18)`、`NUMBER 2 (1:20)`、`MOD (1:22)`、`NUMBER 3 (1:24)`、`EOF`。

语法分析器（递归下降）构建出树：

```

BIN(-) (1:14)
├─ BIN(*) (1:10)
│   └─ UNARY(-) (1:1)
│       └─ BIN(+) (1:5)
│           └─ NUM(2) (1:3)
│               └─ NUM(3) (1:7)
└─ NUM(4) (1:12)
├─ BIN(%) (1:22)
│   └─ BIN(/) (1:18)
│       └─ NUM(6) (1:16)
│           └─ NUM(2) (1:20)
└─ NUM(3) (1:24)

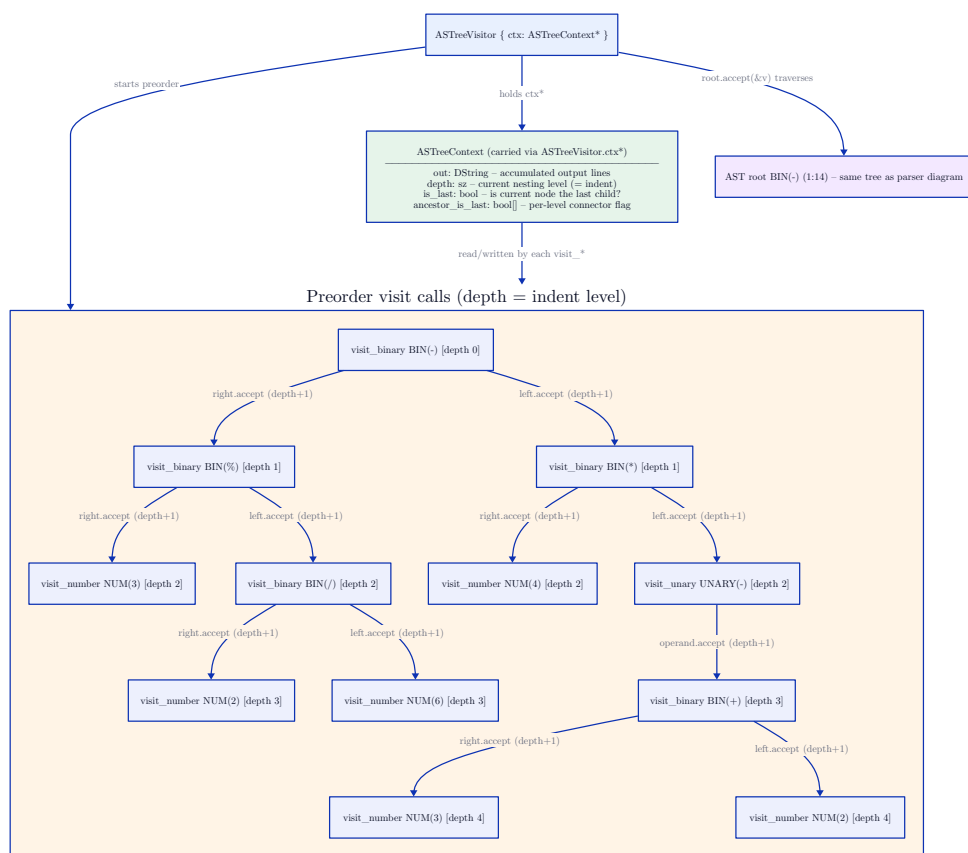
```

节点中的坐标就是对应词法单元的 Ref；它们同样用于错误消息中。

树的遍历上下文（ASTreeVisitor）

to_tree 创建 ASTreeVisitor 并调用 root.accept(&v)。访问者并不将状态保存在调用栈上，而是将其保存在 ASTreeContext 中，并通过 ctx* 引用：

- out: DString —— 累积的结果（树的各行）；
- depth: sz —— 当前的嵌套层级，也就是缩进量；
- is_last: bool —— 当前节点是否为其父节点的最后一个子节点；
- ancestor_is_last: bool[] —— 对每个祖先，标记该祖先是否为其父节点的最后一个子节点；据此选择连接符 | （祖先不是最后一个）或 （最后一个）。



图中展示的是前序遍历（preorder）的顺序。进入 visit_* 时，depth 字段加 1（子节点绘制时缩进更深一级）；退出时减 1。is_last 标志在进入子节点递归前设置：左子树为 false，右子树为 true；节点将自己的值存入 ancestor_is_last[depth-1]，以便子节点绘制正确的前缀。这样，遍历上下文就能在递归调用之间得以保存，而无需通过参数传递。

构建与测试

```
c3c build calc3      # 构建 (debug)
c3c build calc3 -O2 -g0 # 发布 (release)
c3c test calc3       # 74 个单元测试（词法分析器、AST、语法分析器、eval、入口点）
```